

Tech Assignment

The problem we want you to solve

Collections agents spend a lot of time waiting, dialing numbers that don't connect, and doing work that a system should ideally handle for them.

We want to build a **SmartDialer** that improves agent utilization without taking shortcuts on call safety.

There are two basic approaches:

#1 Progressive dialing:

One available agent → one outbound call.

It is predictable and safe, but agents can sit idle.

#2 Predictive dialing:

Start calls before an agent becomes free, based on an estimate of how many borrowers are likely to answer.

It can improve utilization, but it creates a difficult problem:

What happens when more borrowers answer than we have agents available to handle?

For our use case, an abandoned connected call is not just a bad customer experience. It can become a compliance issue.

Your job is to solve one of these problems.

What we want you to build

Build a small but working **SmartDialer functional prototype** with either of the modes.

1. Progressive Mode

If there are 50 available agents, the system should not create more than 50 agent-bound outbound calls.

The system should decide:

- which borrower to call;
- which agent to allocate;
- when the agent becomes reserved;

- what happens when the call fails;
- what happens when the agent disappears during call setup.

2. Predictive Mode

The system should decide how aggressively it can dial.

You can use whatever approach you think makes sense.

For example, you may consider:

- current agent availability;
- calls already connected;
- calls currently ringing;
- historical answer rate;
- call setup time;
- average call duration;
- provider health;
- recent campaign behaviour.

We want to see **how you think about the problem**.

The important part

Your predictive algorithm should never directly place a call.

Put a separate **Safety Controller** between the pacing logic and the telecom provider.

Something along these lines:

Campaign > Pacing Engine (Progressive / Predictive) > Safety Controller > Call Allocator > Telecom Provider

The pacing engine can say:

"I think we can start 15 more calls."

The Safety Controller decides whether that is actually allowed.

It can:

- approve;
- reduce the number;
- reject the request;
- fall back to progressive behaviour.

The predictive algorithm should not have a way to simply switch the safety mechanism off.

Things your system needs to handle

This is where most of the assignment is.

Agent state

Model the agent lifecycle. You don't have to use these exact states, but you should have something equivalent to:

- OFFLINE
- AVAILABLE
- RESERVED
- DIALING
- CONNECTED
- WRAP_UP
- PAUSED

Now assume two workers see the same available agent at almost the same time.

Both must not be able to reserve that agent.

Explain how you prevent it.

Call state

Your call lifecycle should also be explicit.

For example:

- QUEUED
- RESERVED
- INITIATED
- RINGING
- ANSWERED
- CONNECTED
- COMPLETED
- FAILED
- CANCELLED

Now assume the telecom provider sends:

- ANSWERED
- ANSWERED
- ANSWERED
- COMPLETED

Or:

- COMPLETED
- ANSWERED
- RINGING

Or the worker processing the call crashes immediately after **ANSWERED**.

Your system should still end up in a sensible and consistent state.

Don't assume that external systems will behave nicely.

Telecom provider

You don't need a real telecom integration. If you could do that, then that will be the cherry on the cake (Plivio gives free \$10 to build)

Create a simple provider interface and at least **two mock providers**.

Make them behave differently.

For example:

Provider A

- fast;
- reliable;
- low failure rate.

Provider B

- slower;
- occasional timeouts;
- duplicate events;
- events arriving out of order.

Your dialer shouldn't have to know the internal details of either provider.

Also think about what happens if the provider suddenly starts failing.

Distributed system thinking

Assume you eventually have multiple dialer workers:

Worker 1

Worker 2

Worker 3

...

Worker N

All of them may be working on the same campaign.

Think carefully about:

- agent allocation;
- borrower allocation;
- duplicate jobs;
- retries;
- idempotency;
- concurrent updates;
- worker crashes;
- stale state;
- provider events.

You don't have to use Kafka, Redis, microservices, etc.

In fact, **don't add technology just because it sounds impressive.**

If you can solve the problem with a simpler architecture, explain why.

We care about the reasoning behind the architecture.

Failure cases

Your prototype should demonstrate what happens in at least these situations:

1. Worker crash

Agent reserved > Borrower reserved > Call initiated > Worker crashes

What happens when the system comes back?

2. Provider outage

The provider starts timing out.

What happens to:

- existing calls?
- new calls?
- retries?

- pacing?

3. Agent availability suddenly drops

Suppose 100 agents are available and 40 disappear within a few seconds.

How quickly does the dialer react?

4. Duplicate events

The same provider event arrives multiple times.

Does your system create multiple state transitions?

5. Out-of-order events

Events don't arrive in the order you expected.

Does your system break?

Show us the predictive logic

We don't need a fancy ML model.

A sensible statistical or rule-based approach is completely fine.

But we want to understand:

Why did the system decide to make this many calls right now?

Run your simulator with different conditions.

For example:

Scenari	Answer Rate	Avg. Talk Time
A	20%	120 sec
B	50%	90 sec
C	70%	180 sec
D	Changing	Changing

Also introduce provider latency and failures.

Show us what happens to:

- agent utilization;
- calls initiated;
- calls connected;
- pacing;
- safety-controller decisions.

Scale

You don't need to build a system that actually places 10,000 calls per second.

But assume that the system eventually needs to support:

100 agents > 1,000 agents > 10,000 agents

Tell us what you think will break first.

Then tell us how you would fix it.

A good answer isn't:

"Add more servers."

We want to know **where the bottleneck is and why**.

What to submit

Keep the submission practical.

We need:

- working source code;
- README with setup instructions;
- architecture diagram;
- agent state machine;
- call state machine;
- Progressive Dialer;
- Predictive Pacing Engine;
- Safety Controller;
- mock telecom providers;
- tests;
- basic simulation;
- basic load test;
- short architecture decision document.

The system should be reasonably easy for another engineer to run locally.

A small architecture note

We don't care whether you use:

- Go;
- Java;
- Python;
- Rust;
- TypeScript;
- or something else.

We also don't care whether you choose PostgreSQL, MySQL, Redis, Kafka, RabbitMQ, or something completely different.

There is no "correct stack" for this assignment.

We care about whether you can explain:

What did you choose?

Why did you choose it?

What problem does it solve?

What does it make harder?

A simple, well-thought-out system will score better than a complicated system that exists mainly to look impressive.

Vibe Coding

You are allowed to use an assisted coding tool of your choice.

Use ChatGPT, Claude, Cursor, Copilot, Gemini, or whatever tools you normally use.

Vibe coding is fine. We aren't trying to measure how fast you can type boilerplate code.

But there is a condition: **If you submit it, you should be able to explain it.**

During the discussion, we may ask you to:

- walk through the code;
- explain a concurrency decision;
- explain your pacing calculation;
- change part of the implementation;
- debug something;
- explain a failure scenario;
- defend an architectural decision;
- tell us what you would do differently if you had another week.

You don't need to have manually written every line.

You do need to **own the system you submit**.

How we'll evaluate it

We will roughly look at:

Area	Weight
System design	20%
Distributed systems & concurrency	15%
Progressive dialing	10%
Predictive pacing	15%
Safety & correctness	15%
Failure handling	10%
Testing & performance	10%
Code quality & documentation	5%

The order matters.

Correctness comes before cleverness.

A beautiful dashboard doesn't matter if two workers can allocate the same agent.

A sophisticated ML model doesn't matter if the safety boundary can be bypassed.

And 20 microservices don't automatically make a system scalable.

The technical discussion

If the submission looks promising, we'll spend time trying to break it.

Some examples:

Two workers try to reserve the same agent at exactly the same time. Walk us through what happens.

Your database says the agent is AVAILABLE, but your cache says RESERVED. Which one wins?

The provider sends ANSWERED, your worker crashes, and then COMPLETED arrives. What happens?

Your model predicted a 70% answer rate. It suddenly drops to 10%. How does the system protect itself?

We just went from 1,000 to 100,000 agents. What breaks first?

Why did your algorithm decide to initiate 17 calls instead of 10?

What part of your architecture are you least confident about?

There may also be a small live coding/debugging exercise based on your own submission.

Final question

Along with the code, give us a short answer to this:

How would you build a SmartDialer that gets as much of the utilization benefit of predictive dialing as possible, while retaining the deterministic safety characteristics of progressive dialing?

We aren't looking for the longest answer.

We want to see how you think.

Suggested timebox

4–6 hours for the initial assignment.

Don't try to build an entire telecom platform.

Build the important part well.

Think → Design → Build → Break → Fix → Explain.